M. Siri Chandana

2405A41139

1.Valid Parenthesis – LC 20

https://leetcode.com/problems/valid-parentheses/submissions/2189502443/

Description | Accepted | Editorial | Solutions | Submissions

Easy | Topics | Companies | Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

> **Input:** s = "()"
>
> **Output:** true

Example 2:

> **Input:** s = "()[]{}"
>
> **Output:** true

Example 3:

> **Input:** s = "(]"

28.9K | 797

Code

Java | Auto

```java
        return true;*/
        Stack <Character> st = new Stack<>();
        for(int i=0;i<s.length();i++){
            if(s.charAt(i) == '(' || s.charAt(i) == '{' || s.charAt(i) == '['){
                st.push(s.charAt(i));
            }
            else{
                if(!st.isEmpty() && s.charAt(i) == ')' && st.peek()=='('){
                    st.pop();
                }
                else if(!st.isEmpty() && s.charAt(i) == ']' && st.peek()=='['){
                    st.pop();
                }
                else if(!st.isEmpty() && s.charAt(i) == '}' && st.peek()=='{'){
                    st.pop();
                }
                else return false;
            }
        }
```

Saved | Ln 1, Col

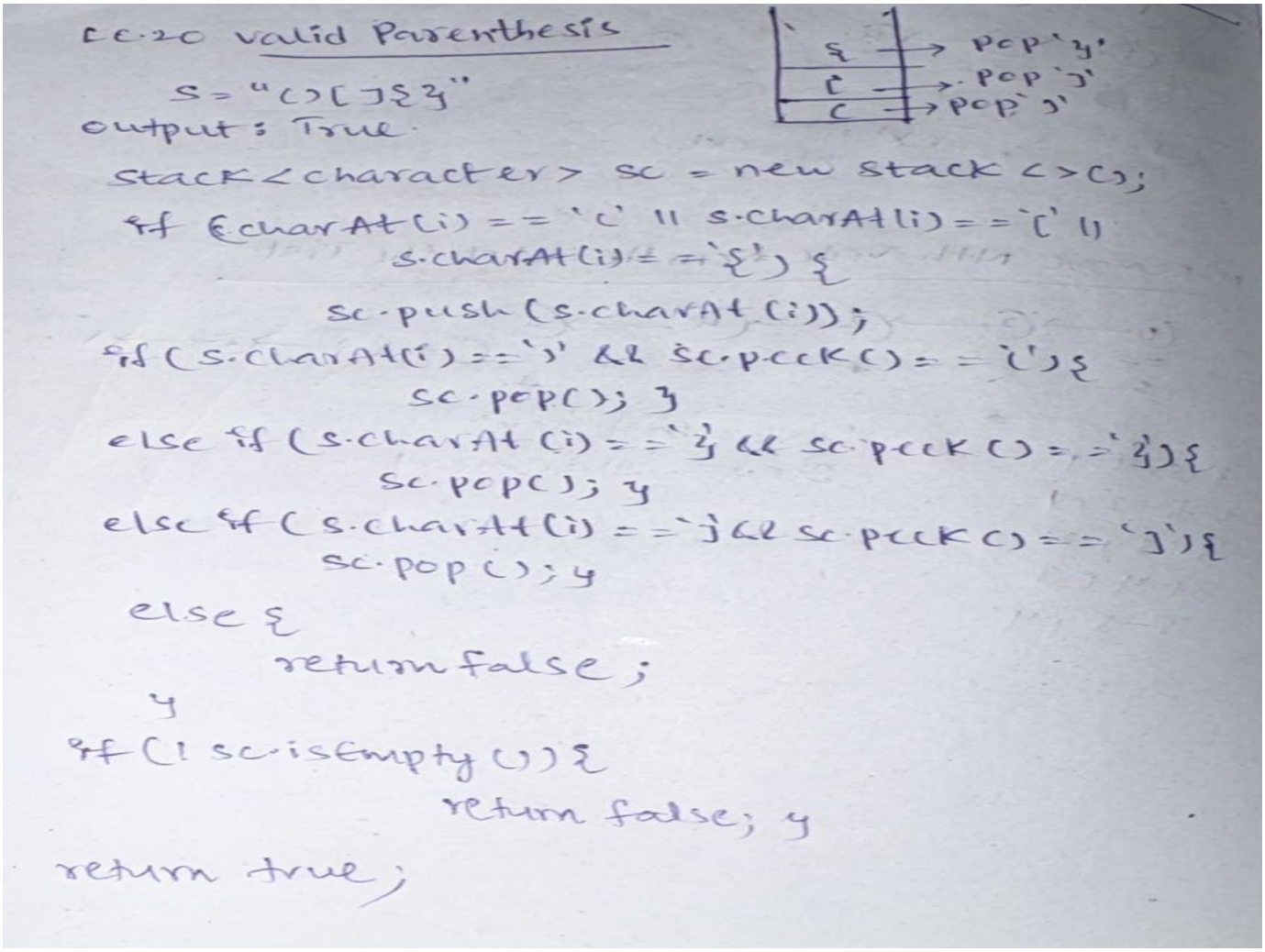
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3 | Case 4 | Case 5

Input

s =



2. Min Stack – LC 155

https://leetcode.com/problems/min-stack/submissions/2189508507/

Description | Accepted | Editorial | Solutions | Submissions

155. Min Stack

Solved

Medium | Topics | Companies | Hint

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int value)` pushes the element `value` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with `O(1)` time complexity for each function.

Example 1:

Input
```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]
```

Output
```
[null,null,null,null,-3,null,0,-2]
```

Explanation
```
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
```

16.4K | 357

</> Code

Java | Auto

```
15      public void push(int val) {
16          s1.push(val);
17          if(s2.isEmpty() || val <= s2.peek()) {
18              s2.push(val);
19          }
20      }
21      public void pop() {
22          if(s1.peek().equals(s2.peek())) {
23              s2.pop();
24          }
25          s1.pop();
26      }
27      public int top() {
28          return s1.peek();
29      }
30      public int getMin() {
31          return s2.peek();
32      }
33  }
34
```

Saved

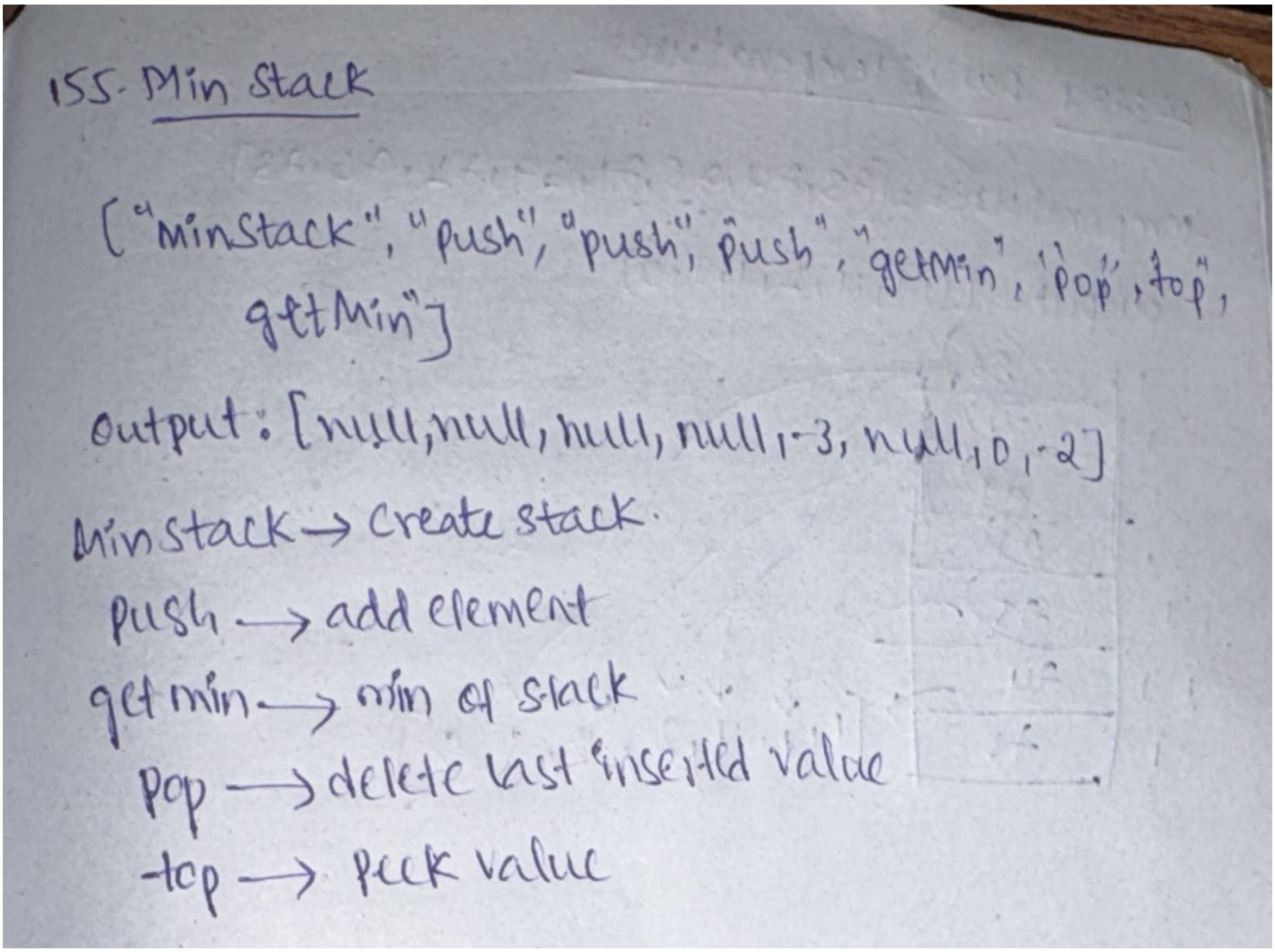
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]
```



3. Next greater element – LC 496

https://leetcode.com/problems/next-greater-element-i/submissions/2189514642/

Description | Accepted | Editorial | Solutions | Submissions

496. Next Greater Element I

Solved

Easy | Topics | Companies

The **next greater element** of some element `x` in an array is the **first greater** element that is **to the right** of `x` in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each `0 <= i < nums1.length`, find the index `j` such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return *an array* `ans` *of length* `nums1.length` *such that* `ans[i]` *is the* ***next greater element*** *as described above.*

Example 1:

```
Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
Output: [-1,3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
- 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3.
- 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
```

Example 2:

```
Input: nums1 = [2,4], nums2 = [1,2,3,4]
Output: [3,-1]
Explanation: The next greater element for each value of nums1 is as
```

Code

Java | Auto

```java
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        int[] ans = new int[nums1.length];
        for(int i = 0; i < nums1.length; i++) {
            int j = 0;
            while(j < nums2.length && nums2[j] != nums1[i]) {
                j++;
            }
            int k = j + 1;
            while(k < nums2.length && nums2[k] < nums1[i]) {
                k++;
            }
            if(k < nums2.length) {
                ans[i] = nums2[k];
            }
            else {
                ans[i] = -1;
            }
        }
        return ans;
    }
}
```

Saved

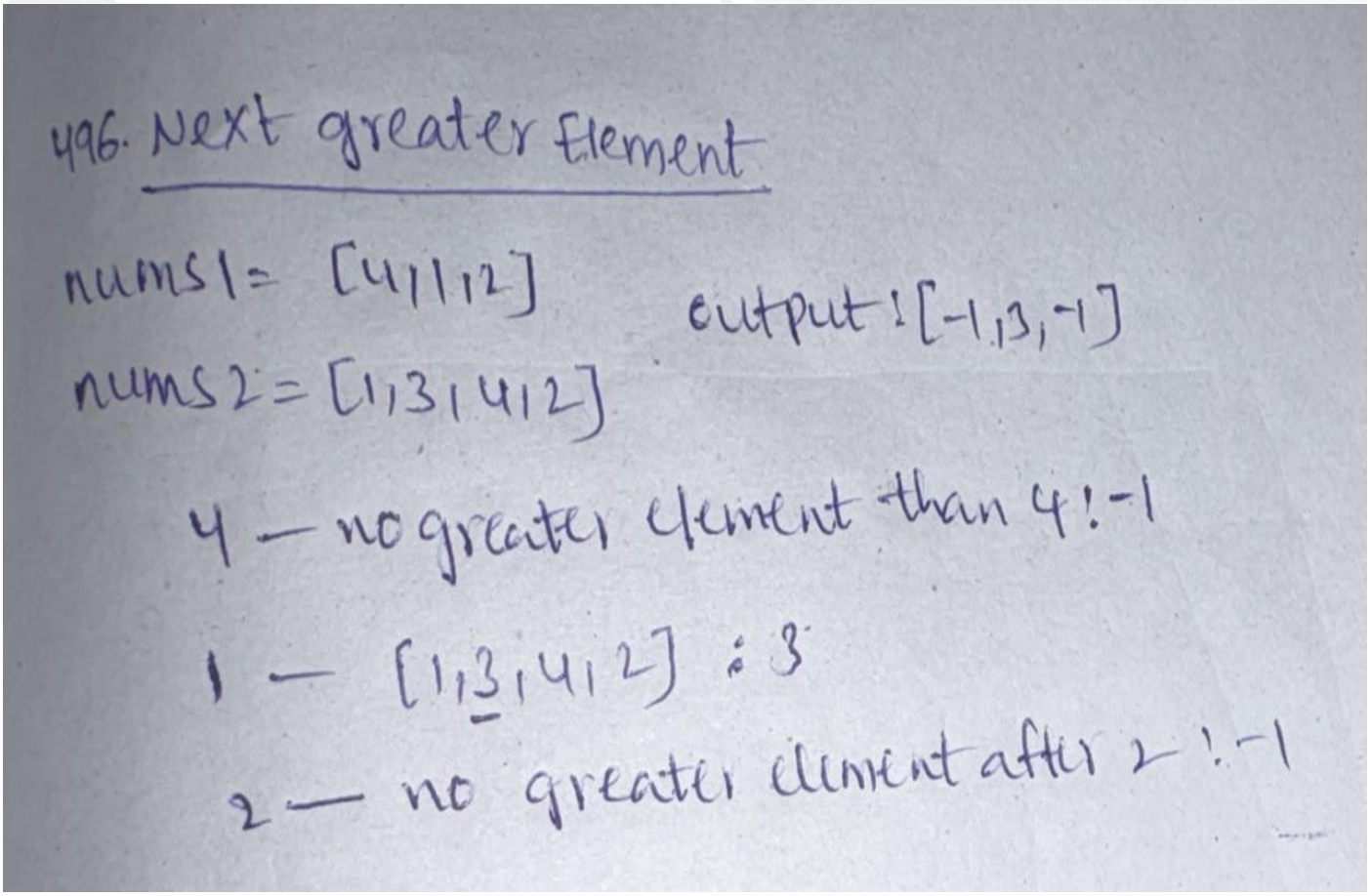
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums1 =
[4,1,2]



4. Evaluate reverse polish nation – LC 150

https://leetcode.com/problems/evaluate-reverse-polish-notation/submissions/2189521048/

Description | Accepted | Editorial | Solutions | Submissions

150. Evaluate Reverse Polish Notation

Solved

Medium | Topics | Companies

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Example 1:

```
Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
```

Example 2:

```
Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6
```

Example 3:

Code

Java | Auto

```java
class Solution {
    public int evalRPN(String[] tokens) {
        Stack<Integer> s = new Stack<>();
        for(int i = 0; i < tokens.length; i++) {
            String ch = tokens[i];
            if(ch.equals("+") || ch.equals("-") ||
               ch.equals("*") || ch.equals("/")) {
                int b = s.pop();
                int a = s.pop();
                if(ch.equals("+")) {
                    s.push(a + b);
                }
                else if(ch.equals("-")) {
                    s.push(a - b);
                }
                else if(ch.equals("*")) {
                    s.push(a * b);
                }
                else if(ch.equals("/")) {
                    s.push(a / b);
                }
            }
            else {
                s.push(Integer.parseInt(ch));
```

Saved | Upgrade to Cloud Saving

Testcase | Test Result

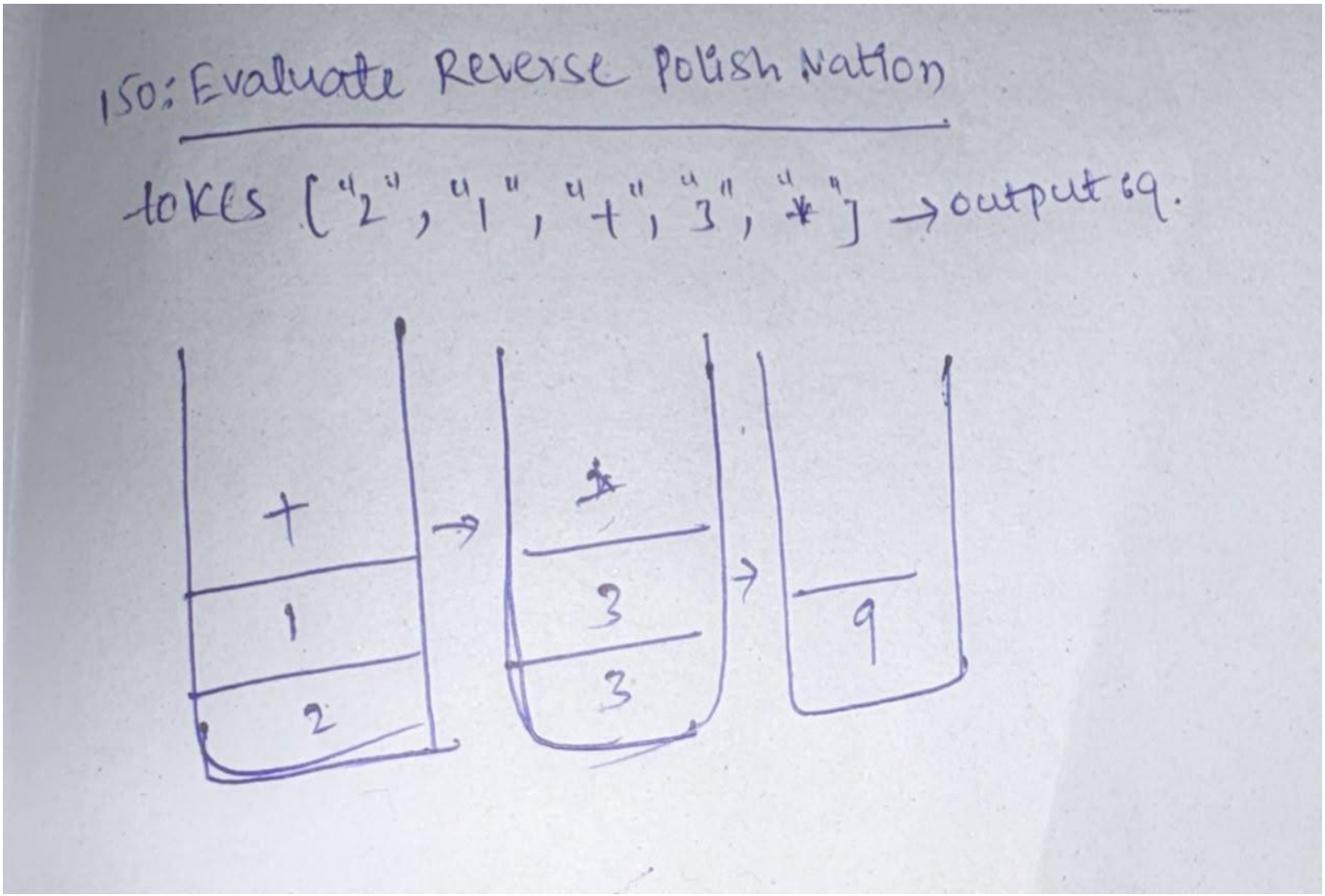
Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

tokens =

["2","1","+","3","*"]



5. Daily Temperatures – LC 739

https://leetcode.com/problems/daily-temperatures/submissions/2189527438/

Description | Accepted | Editorial | Solutions | Submissions

739. Daily Temperatures

Solved

Medium | Topics | Companies | Hint

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i`th *day to get a warmer temperature.* If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

```
Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]
```

Example 2:

```
Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]
```

Example 3:

```
Input: temperatures = [30,60,90]
Output: [1,1,0]
```

Constraints:

- `1 <= temperatures.length <= 10^5`
- `30 <= temperatures[i] <= 100`

Code

Java | Auto

```java
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        Stack<Integer> st = new Stack<>();
        int n = temperatures.length;
        int [] arr = new int[n];
        /*int count = 0;
        for(int i=n-1;i>=0;i--){
            while(!st.isEmpty() && st.peek() <= temperatures[i]){
                st.pop();
                count++;
            }
            if(!st.isEmpty() && st.peek()>temperatures[i]){
                arr[i] = count;
            }
            else if(st.isEmpty()){
                st.push(arr[i]);
            }
        }
        return arr;*/
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && temperatures[st.peek()] <= temperatures[i]) {
                st.pop();
            }
            if (!st.isEmpty()) {
```

Saved | Ln 28, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

